

שבוע תשע על עמוד אחד

שבוע: 9

המשפט אחד של השבוע

אם זה על המסך — זה הגיע מ- `state` . אם זה לא ב- `state` — זה לא קיים.

המעגל החד־כיווני

`state` נכנס ל- `render` , מצייר את המסך, המשתמש פועל, המטפל משנה את `state` . אין חץ חוזר מהמסך אל `state` . זה השם שנשתמש בו עד סוף הסמסטר, וזו הצורה שהמבחן בשבוע 13 כתוב מולה.

שלושת החוקים

#	החוק	מה הוא אוסר
1	אמת אחת	שני מקומות שבהם אותה עובדה כתובה
2	צייר אחד	משהו שאינו <code>render</code> שכותב למסך
3	כיוון אחד	קוד שקורא את המסך כדי לדעת מה נכון

ומהם נובע הרביעי, וזה זה שנופלים בו: כל מה שאפשר לגזור — נגזר.

שלוש השכבות, ומה אסור לכל אחת

הקובץ	התפקיד	מה אסור לו
<code>js/state.js</code>	האובייקט וכל הגזירות	לגעת ב-DOM
<code>js/render.js</code>	<code>state</code> נכנס, מסך יוצא	לשנות <code>state</code> · לרשום מאזינים · לקרוא את המסך
<code>js/events.js</code>	פעולת משתמש הופכת לשינוי <code>state</code>	לצייר
<code>js/app.js</code>	לחבר את השלושה, פעם אחת	לגדול

העמודה השלישית היא זו שכדאי לצלם. שכבה מוגדרת לפי מה שאסור לה, לא לפי מה שיש בה.

בדיקה עצמית בשלושים שניות

חפש `document` ב- `state.js` . חפש `createElement` ב- `events.js` . אם מצאת — משהו זלג.

ארבע השורות של `app.js`

#	מה	למה בסדר הזה
1	להכניס את הנתונים ל- <code>state</code>	לפני שמישהו מצייר
2	<code>subscribe(render)</code>	המסך עוקב אחרי כל שינוי, מעכשיו
3	<code>wire()</code>	פעם אחת, ולא מתוך <code>render</code>
4	<code>render(getState())</code>	זו שנסכחת. ההרשמה נרשמת לשינויים הבאים

אם העמוד ריק: הקלד תו בתיבת החיפוש. אם הרשימה קופצת — שורה 4 חסרה, וה- `render` שלך מצוין.

חמש הטעויות שהשבוע נועד למנוע

#	הטעות	הסימן	התיקון
1	לעדכן את ה-DOM ואת המערך בנפרד	פריט שהוסר חוזר בהוספה הבאה	המטפל קורא ל- <code>setState</code> בלבד
2	לשמור משהו שאפשר לגזור	עובד, עד שמישהו משנה את האוסף במקום אחר	פונקציית <code>*select</code> ב- <code>state.js</code>
3	אינדקס כזהות	משנה את הפריט הלא נכון, ומראה את הנכון משתנה	<code>data-id</code> על השורה
4	מטפל שמצייר	שלושה דברים לזכור בכל מטפל, לנצח	<code>setState</code> , וזהו
5	קובץ אחד ארוך	כל שינוי מחייב לקרוא את כולו	שלוש שכבות

הגזירה שהיא כל הפיצ'ר

חיפוש, סינון ומיין אינם שלוש התנהגויות. הם **שדות ב-`state`** ופונקציה אחת שמחשבת מהם מה מוצג. לכן שלושת המטפלים הם **שורה אחת כל אחד** — והם לא מסננים ולא ממיינים.

מטפל מיין שאורכו יותר משורה אחת ממין משהו, ומיין הוא גזירה.

שני דברים לזכור בגזירה עצמה:

- `filter` מחזיר מערך חדש, ולכן ה-`sort` שאחריו בטוח. מיין ישיר של `state.items` הוא פקד תצוגה שעורך את הנתונים.
- להשוות מחרוזות עם `localeCompare` ולא עם `<`. השני משווה קודי תווים.

שלושה עמודים, `render` אחד — ואיפה יושב המזהה

כל אזור ב- `render` יוצא מיד אם נקודת העיגון שלו אינה בעמוד, ולכן **אותו קובץ בדיוק** משרת את שלושת העמודים ואין בו תנאי "באיזה עמוד אנחנו".

עמוד הפריט צריך לדעת **איזה** פריט, והמקום היחיד שבו זה כתוב הוא ה-URL:

```
const id = new URLSearchParams(location.search).get('id'); // item.html?id=42 gives "42"
```

שלושה דברים שכדאי לדעת עליו, כי הוא חוזר בפרויקט ובשבוע 11:

- **הוא מחזיר מחרוזת, תמיד** – או `null` אם הפרמטר אינו שם. `42 === "42"` הוא `false`, אז אם המזהים שלך מספרים, המרה היא באחריותך ובנקודה אחת.
- **`null` הוא מצב ריק, לא שגיאה**. `item.html?id=nope` מציג משפט ודרך חזרה.
- **הכיוון ההפוך – לבנות כתובת – הוא `new URL` ו-`url.searchParams.set`**, ואותו רואים בשבוע 11 מול API אמיתי. אותו אובייקט, אותו מנגנון בריחה, שני כיוונים.

השורה הזאת כתובה עבורך בתיקיית הפתיחה. היא כאן כדי שתדע מה היא עושה כשתראה אותה בפרויקט שלך.

רכיבים

רכיב הוא **פונקציה: נתון נכנס, סימון יוצא, בלי תופעות לוואי**. תבנית מחרוזת ו-`createElement` שתיהן דרכים לגיטימיות, והן מייצרות אותו סימון בדיוק.

מה שלא לגיטימי הוא טקסט שמשתמש הקליד בלי בריחה. `esc` הוא שש שורות, וסדר ההחלפות בו אינו קוסמטי: **& ראשון**, אחרת אתה בורח מהבריחות של עצמך.

ביצועים, בכנות

רינדור מלא של כמה מאות שורות הוא בתוך פריים אחד. **מה שנשבר ראשון אינו המהירות** – הוא הפוקוס: פקד שנמצא בתוך האזור שמצויר מחדש נמחק תוך כדי שמקלידים בו.

הכלל: שום פקד שהמשתמש אוחז בו לא נמצא בתוך האזור שאתה מצייר מחדש.

שמונה ההדגמות

#	התיקיה	הרעיון האחד
01	<code>demos/01-two-truths</code>	שני מקומות, ומרחק שרק גדל
02	<code>demos/02-patch-vs-redraw</code>	אותו יישום, מטליא מול מצייר מחדש
03	<code>demos/03-render-is-a-function</code>	ערוך את האובייקט, המסך עוקב
04	<code>demos/04-components</code>	אותה שורה בארבע דרכים, ומה שרץ
05	<code>demos/05-identity</code>	מיקום אינו זהות
06	<code>demos/06-derived-vs-stored</code>	כמה זמן תשובה שמורה נשאת נכונה
07	<code>demos/07-three-layers</code>	כמה שורות צריך לקרוא

#	התיקיה	הרעיון האחד
08	demos/08-rerender-cost	המספרים של המחשב שלך

כולן מודדות את מה שהן טוענות. אפשר להריץ אותן בבית ולהתווכח עם הדפדפן.

מה נשמר לשבוע הבא

ה-state הזה חי בזיכרון ונעלם ברענון. בשבוע 10 הוא מפסיק להיעלם — וזה שינוי **בקובץ אחד**, כי הצורה כבר נכונה.